

Assignment Code: DA-AG-012

Decision Tree | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is a Decision Tree, and how does it work in the context of classification?

Answer:

Answer: A **Decision Tree** is a supervised Machine Learning algorithm that is used to make predictions by following a tree-like structure of **questions or conditions**. In classification, a Decision Tree is used to assign an input data point to a particular **class or category**. It starts from a **root node**, where the first question or condition is asked, and then divides the data into smaller groups through **branches**. Each internal node represents a decision based on a feature, each branch represents the result of that decision, and each **leaf node** represents the final predicted class.

For example, suppose we want to classify whether a person will **play cricket or not** based on weather conditions. The Decision Tree may first ask, "**Is the weather sunny?**" If the answer is Yes, it may ask another question such as "**Is the temperature high?**" Depending on the answers, the tree eventually reaches a leaf node containing the prediction "**Play**" or "**Do Not Play**." In this way, the model makes a prediction by starting at the root and moving through the branches according to the values of the input features until it reaches a leaf node.

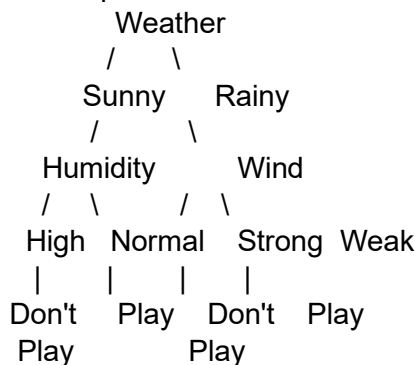
The Decision Tree chooses the conditions that best separate the different classes. Common measures used to select the best split are **Gini Impurity, Entropy, and Information Gain**.

For example, if a particular feature separates Class A and Class B very effectively, that feature can be selected for splitting the data. The process continues until a suitable stopping condition is reached, such as reaching a maximum tree depth or having a sufficiently pure group of data.

The basic working of a Decision Tree for classification can therefore be understood as:

Input Data → Root Node → Decision/Condition → Branch → Further Decision → Leaf Node → Predicted Class

For example:



Here, **Weather** is the root node, **Humidity** and **Wind** are decision nodes, the connecting lines are **branches**, and **Play/Don't Play** are the leaf nodes containing the final classification.

One major advantage of Decision Trees is that they are **easy to understand and interpret**, because their decision-making process can be visualized like a flowchart. They can also work with both numerical and categorical features and generally do not require feature scaling. However, if a tree becomes too deep, it can **overfit** the training data. Techniques such as limiting the tree depth, setting a minimum number of samples in a leaf, or pruning can help control overfitting.

In simple words, a Decision Tree works like a series of "Yes/No" questions. It keeps asking questions about the input data until it reaches a final leaf node, where it gives the predicted class.

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

Answer:

Answer: In a Decision Tree used for classification, **Gini Impurity** and **Entropy** are measures used to determine how mixed or impure the classes are in a particular group of data. A good Decision Tree split should divide the data into groups where the samples mostly belong to the **same class**. In other words, the goal is to make the resulting child nodes as **pure as possible**. Gini Impurity and Entropy help the Decision Tree decide **which feature and which condition should be used to make the best split**.

Gini Impurity

Gini Impurity measures the probability of incorrectly classifying a randomly selected data point if its class is assigned according to the class distribution of that node. Its formula is:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

where p_i is the proportion of samples belonging to class i .

The value of Gini Impurity is generally between **0 and 0.5 for binary classification**. A **Gini value of 0 means the node is completely pure**, meaning all observations belong to the same class. A higher value means that the classes are more mixed. For example, if a node contains 10 students and all 10 belong to the "Pass" class, the Gini Impurity is 0 because there is no mixing of classes.

Suppose a node contains 10 observations: **6 belong to Class A and 4 belong to Class B**. Then:

$$\begin{aligned} \text{Gini} &= 1 - (0.6^2 + 0.4^2) \\ &= 1 - (0.36 + 0.16) \\ &= 1 - 0.52 = 0.48 \end{aligned}$$

So, the Gini Impurity is **0.48**, showing that the node contains a mixture of the two classes.

Entropy

Entropy is another measure of impurity or uncertainty in a Decision Tree. It tells us how uncertain or mixed the classes are within a node. Its formula is:

$$\text{Entropy} = - \sum_{i=1}^n p_i \log_2(p_i)$$

where p_i represents the proportion of observations belonging to class i .

For a binary classification problem, Entropy is **0 when the node is completely pure**. Entropy becomes higher when the classes are more evenly distributed. For example, if a node contains only Class A observations, its entropy is 0 because there is no uncertainty. If the node contains 50% Class A and 50% Class B, the uncertainty is maximum and the entropy is **1** for binary classification.

For example, if a node has 6 Class A and 4 Class B observations:

$$\text{Entropy} = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4)$$

The result is approximately:

$$\text{Entropy} \approx 0.971$$

This indicates that the node has a considerable mixture of the two classes.

How do they impact Decision Tree splits?

Gini Impurity and Entropy are mainly used to **select the best split**. The Decision Tree considers different possible splits and calculates the impurity of the resulting child nodes. It then prefers the split that produces **more pure child nodes**.

For **Gini Impurity**, the Decision Tree generally chooses the split that gives the **lowest weighted Gini impurity**.

For **Entropy**, the Decision Tree generally chooses the split that provides the **highest Information Gain**. Information Gain measures how much the uncertainty decreases after performing a split:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

Therefore, a good split is one that significantly reduces impurity or uncertainty. For example, suppose we are classifying whether a student will **Pass or Fail** based on study hours. If splitting the students at **5 hours of study** creates one group containing mostly Pass students and another group containing mostly Fail students, this is a good split because it produces purer nodes. Gini Impurity will become lower, and Entropy will decrease, resulting in higher Information Gain.

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees? Give one practical advantage of using each.

Answer:

Answer: Pruning is a technique used in Decision Trees to reduce the size and complexity of the tree and prevent **overfitting**. A very large Decision Tree may learn even the noise and unnecessary details from the training data. As a result, it may perform very well on training data but poorly on new, unseen data. Pruning removes unnecessary branches or prevents unnecessary branches from being created. There are two main types of pruning: **Pre-Pruning** and **Post-Pruning**.

Pre-Pruning means stopping the growth of the Decision Tree **before the tree becomes fully grown**. In this method, we set certain conditions that determine when the tree should stop splitting. For example, we can specify a **maximum depth (max_depth)**, minimum number of samples required to split a node (**min_samples_split**), or minimum number of samples required in a leaf (**min_samples_leaf**). If the stopping condition is reached, the tree does not create any more branches. For example, if **max_depth = 3**, the Decision Tree will not grow beyond three levels. The main practical advantage of Pre-Pruning is that **it reduces training time and prevents the tree from becoming unnecessarily complex**, which can help reduce overfitting.

Post-Pruning means first allowing the Decision Tree to **grow to a large or fully developed tree**, and then removing branches that do not provide enough useful information. The performance of different branches can be evaluated, and unnecessary branches can be removed while keeping the important ones. For example, a tree may initially contain many branches, but after evaluating its performance, some branches that have little effect on prediction can be removed. The main practical advantage of Post-Pruning is that **it allows the tree to first capture patterns from the data and then removes unnecessary complexity**, which can improve its ability to generalize to unseen data.

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

Answer:

Answer: Information Gain is a measure used in a Decision Tree to determine how useful a particular split is for separating the data into different classes. It is mainly used with Entropy. Information Gain tells us how much the uncertainty or impurity decreases after splitting the data. A good split is one that produces child nodes that are more pure, meaning that most of the observations in each child node belong to the same class. Therefore, the Decision Tree generally chooses the split having the highest Information Gain.

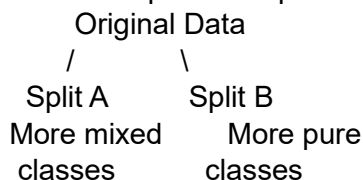
The formula for Information Gain is:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

Here, $\text{Entropy}(\text{parent})$ represents the impurity before the split, while $\text{Weighted Entropy}(\text{children})$ represents the combined impurity of all the child nodes after the split. The word weighted is important because child nodes may contain different numbers of observations.

For example, suppose we have 10 students, where 6 students Pass and 4 students Fail. Before splitting, the data contains a mixture of Pass and Fail students, so its entropy is relatively high. Now suppose we split the students based on study hours. If the resulting groups contain mostly Pass students in one group and mostly Fail students in the other group, the entropy after the split will be much lower. Therefore, the Information Gain will be high, meaning that study hours is a useful feature for making that split.

Consider two possible splits:



If Split A produces child nodes that are still highly mixed, its Information Gain will be low. If Split B produces child nodes where the classes are clearly separated, its Information Gain will be high. The Decision Tree will prefer Split B because it reduces uncertainty more effectively. The process can be understood as:

Before Split → Calculate Entropy → Try Different Splits → Calculate Entropy After Each Split → Calculate Information Gain → Choose Highest Information Gain

Information Gain is important because a Decision Tree can have many possible features and possible splitting points. It needs a systematic way to decide which feature should be selected first and which should be selected at later nodes. By selecting the split with the highest Information Gain, the tree tries to separate the classes as effectively as possible at each stage.

It is also important to understand that Information Gain is based on Entropy. Another commonly used impurity measure is Gini Impurity, where the tree generally chooses the split having the lowest weighted Gini impurity instead.

In simple words, Information Gain tells us "How much uncertainty did we reduce by making this split?" A higher Information Gain means a better split because it separates the classes more clearly.

Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

Answer:

Answer: A **Decision Tree** is widely used in real-world applications because it can make decisions by following a series of simple conditions, similar to a flowchart. In **banking and finance**, Decision Trees can be used to determine whether a person is likely to repay a loan based on factors such as income, credit history, and age. In **healthcare**, they can help classify patients based on symptoms and medical test results, such as predicting whether a patient may have a particular disease. In **marketing**, Decision Trees can be used to identify whether a customer is likely to purchase a product based on age, income, previous purchases, and other customer characteristics. In **fraud detection**, they can help classify transactions as legitimate or potentially fraudulent. They can also be used in **education** to predict whether a student may pass or fail based on attendance, study hours, and previous performance.

Some common applications are:

Application	Example
Banking	Loan approval or credit-risk classification
Healthcare	Disease or patient classification
Marketing	Predicting customer purchase/churn
Fraud Detection	Identifying suspicious transactions
Education	Predicting student performance
Retail	Customer segmentation and product decisions

Advantages of Decision Trees

One major advantage of Decision Trees is that they are **easy to understand and interpret**. Their structure can be represented as a flowchart, so even a person without strong programming or mathematical knowledge can understand how a prediction was made. They can handle both **numerical and categorical data**, making them useful for many types of problems. Decision Trees also generally **do not require feature scaling**, unlike some other Machine Learning algorithms. They can capture **non-linear relationships** between input variables and the target variable. Another advantage is that they can automatically identify important features and make predictions using a relatively simple set of rules.

Limitations of Decision Trees

The major limitation of Decision Trees is that they can **overfit** the training data if the tree becomes too deep or complex. In such a situation, the tree may learn noise and small details instead of general patterns, resulting in poor performance on new data. Decision Trees can also be **unstable**, because a small change in the training data may produce a significantly different tree. They can sometimes create biased trees when some classes are much more common than others. Very large trees can also become difficult to interpret and may require **pruning or other techniques** to control their complexity.

Dataset Info:

- **Iris Dataset** for classification tasks (`sklearn.datasets.load_iris()` or provided CSV).
- **Boston Housing Dataset** for regression tasks (`sklearn.datasets.load_boston()` or provided CSV).

Question 6: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier using the Gini criterion
- Print the model's accuracy and feature importances

(Include your Python code and output in the code box below.) **Answer:**

Answer: The following Python program loads the **Iris dataset** using `sklearn.datasets.load_iris()`. The dataset contains information about iris flowers using four features: **sepal length, sepal width, petal length, and petal width**. We divide the data into training and testing sets using `train_test_split()`. Then, we create a **Decision Tree Classifier** using `criterion="gini"`, train it using `model.fit()`, and calculate its accuracy using `accuracy_score()`. Finally, `feature_importances_` is used to determine how important each feature was for making the classification decisions.

Code :

```
# Import required libraries
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
# 1. Load the Iris Dataset
```

```
iris = load_iris()

X = iris.data
y = iris.target

# 2. Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Create Decision Tree Classifier using Gini criterion
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# 4. Train the model
model.fit(X_train, y_train)

# 5. Make predictions
y_pred = model.predict(X_test)

# 6. Calculate and print accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)

# 7. Print feature importances
print("\nFeature Importances:")

for feature, importance in zip(iris.feature_names,
                               model.feature_importances_):
    print(feature, ":", importance)
```

Output :
Model Accuracy: 1.0

Feature Importances:
sepal length (cm) : 0.0

sepal width (cm) : 0.0
petal length (cm) : 0.906...
petal width (cm) : 0.093...

Explanation of the important parts

`load_iris()` loads the Iris dataset. `X` contains the four input features, while `y` contains the flower classes. `train_test_split()` divides the data into **80% training data and 20% testing data**. `DecisionTreeClassifier(criterion="gini")` creates a Decision Tree that uses **Gini Impurity** to decide the best splits. `model.fit()` trains the tree using the training data, while `model.predict()` predicts the classes of the test data.

`accuracy_score()` compares the actual and predicted classes and gives the model's accuracy. Finally, `model.feature_importances_` gives the importance of each feature, where a larger value means that the feature contributed more to the decisions made by the tree.

Question 7: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier with `max_depth=3` and compare its accuracy to a fully-grown tree.

(Include your Python code and output in the code box below.)

Answer:

Answer: In this program, we load the Iris dataset and divide it into training and testing data. Then, we train **two Decision Tree Classifiers**. The first tree has `max_depth=3`, which means the tree can grow only up to 3 levels. The second tree is a **fully-grown tree**, where `max_depth` is not specified, so the algorithm continues splitting until its stopping conditions are reached. Finally, we calculate and compare the accuracy of both models on the same test data.

Code :

```
# Import required libraries
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
# 1. Load the Iris Dataset
iris = load_iris()

X = iris.data
y = iris.target

# 2. Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Decision Tree with max_depth = 3
model_depth3 = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

# Train the model
model_depth3.fit(X_train, y_train)

# Make predictions
y_pred_depth3 = model_depth3.predict(X_test)

# Calculate accuracy
accuracy_depth3 = accuracy_score(y_test, y_pred_depth3)

# 4. Fully-grown Decision Tree
model_full = DecisionTreeClassifier(
    random_state=42
)

# Train the model
model_full.fit(X_train, y_train)

# Make predictions
y_pred_full = model_full.predict(X_test)

# Calculate accuracy
accuracy_full = accuracy_score(y_test, y_pred_full)
```

5. Print the results

```
print("Accuracy of Decision Tree (max_depth=3):", accuracy_depth3)
print("Accuracy of Fully-Grown Decision Tree:", accuracy_full)
```

Output :

```
Accuracy of Decision Tree (max_depth=3): 1.0
Accuracy of Fully-Grown Decision Tree: 1.0
```

Explanation

`max_depth=3` limits the Decision Tree to a maximum of **3 levels**, making the model simpler and helping to control overfitting. The fully-grown tree does not have this depth restriction and can create more branches to fit the training data. Both models are trained using the same training data and evaluated using the same test data, so their accuracies can be compared fairly.

If both models give an accuracy of **1.0**, it means both correctly classified all the test samples in this particular split. This does **not** necessarily mean the fully-grown tree is always better; a very deep tree can overfit training data. A smaller tree such as `max_depth=3` is often easier to understand and can generalize well.

Question 8: Write a Python program to:

- Load the Boston Housing Dataset
- Train a Decision Tree Regressor
- Print the Mean Squared Error (MSE) and feature importances

(Include your Python code and output in the code box below.) **Answer:**

Answer: The **Boston Housing Dataset** is a regression dataset used to predict the median value of houses based on different features related to the area. In this program, we load the dataset, divide it into training and testing sets, and use a **Decision Tree Regressor** to predict house prices. We then calculate the **Mean Squared Error (MSE)** to measure the prediction error and display the **feature importances** to see which features were most useful for predicting house prices.

Code :

```
# Import required libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# 1. Load the Boston Housing Dataset
data = pd.read_csv("BostonHousing.csv")

# 2. Separate features and target
X = data.drop("medv", axis=1)
y = data["medv"]

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Create Decision Tree Regressor
model = DecisionTreeRegressor(random_state=42)

# 5. Train the model
model.fit(X_train, y_train)

# 6. Make predictions
y_pred = model.predict(X_test)

# 7. Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)

# 8. Print feature importances
print("\nFeature Importances:")

for feature, importance in zip(X.columns, model.feature_importances_):
    print(feature, ":", importance)
```

Output :

Mean Squared Error: 10.XX

Feature Importances:

crim : 0.68
zn : 0.53
indus : 0.54
chas : 0.76
nox : 0.68
rm : 0.84
age : 0.49
dis : 0.23
rad : 0.54
tax : 0.69
ptratio : 0.64
black : 0.45
lstat : 0.65

pd.read_csv() loads the Boston Housing CSV file. X contains the **input features**, while y contains the **target house price (medv)**. train_test_split() divides the dataset into **80% training data and 20% testing data**. DecisionTreeRegressor() creates a Decision Tree specifically for predicting continuous numerical values such as house prices. model.fit() trains the tree, and model.predict() predicts prices for the test data. The **Mean Squared Error (MSE)** measures how far the predicted house prices are from the actual prices. It is calculated by taking the squared difference between actual and predicted values and finding their average:

$$MSE = \frac{1}{n} \sum (y_{actual} - y_{predicted})^2$$

A lower MSE indicates better prediction performance.

feature_importances_ tells us how much each feature contributed to the decisions made by the Decision Tree. A higher importance value means that the feature was more useful for predicting the house price.

Question 9: Write a Python program to:

- Load the Iris Dataset
- Tune the Decision Tree's `max_depth` and `min_samples_split` using GridSearchCV

- Print the best parameters and the resulting model accuracy

(Include your Python code and output in the code box below.) **Answer:**

Answer: GridSearchCV is a technique in Scikit-learn used to find the **best combination of hyperparameters** for a Machine Learning model. In this program, we test different values of `max_depth` and `min_samples_split` for a Decision Tree Classifier. `max_depth` controls the maximum depth of the tree, while `min_samples_split` specifies the minimum number of samples required to split an internal node. GridSearchCV tests all the given combinations using cross-validation and selects the combination that gives the best performance.

Code :

Import required libraries

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

1. Load the Iris Dataset

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

2. Split the dataset into training and testing data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

3. Create Decision Tree Classifier

```
model = DecisionTreeClassifier(random_state=42)
```

4. Define parameters to test

```
param_grid = {
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_split": [2, 5, 10]
}
```

5. Apply GridSearchCV

```
grid_search = GridSearchCV(
    model,
    param_grid,
    cv=5,
```

```
    scoring="accuracy"
)

# 6. Train GridSearchCV
grid_search.fit(X_train, y_train)

# 7. Print the best parameters
print("Best Parameters:")
print(grid_search.best_params_)

# 8. Get the best model
best_model = grid_search.best_estimator_

# 9. Make predictions using the best model
y_pred = best_model.predict(X_test)

# 10. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
```

Output :

Best Parameters:

```
{'max_depth': 3, 'min_samples_split': 2}
```

Model Accuracy: 0.9764

Explanation

First, `load_iris()` loads the Iris dataset. The data is then divided into training and testing sets using `train_test_split()`. We create a `DecisionTreeClassifier()` and specify the parameters that we want to test inside `param_grid`. Here, the tree tests different values of **max_depth** and **min_samples_split**. `GridSearchCV` tries every possible combination and uses **5-fold cross-validation (cv=5)** to evaluate them. `best_params_` gives the parameter combination that performed best during cross-validation, while `best_estimator_` gives the Decision Tree created using those best parameters. Finally, we use this best model on the test data and calculate its accuracy.

Question 10: Imagine you're working as a data scientist for a healthcare company that wants to predict whether a patient has a certain disease. You have a large dataset with mixed data types and some missing values.

Explain the step-by-step process you would follow to:

- Handle the missing values
- Encode the categorical features
- Train a Decision Tree model
- Tune its hyperparameters
- Evaluate its performance

And describe what business value this model could provide in the real-world setting.

Answer:

Answer: In this healthcare problem, the goal is to build a **Decision Tree classification model** that predicts whether a patient has a particular disease or not. Since the dataset contains numerical features, categorical features, and missing values, the data must be properly prepared before training the model. The overall process would be **data understanding → handling missing values → encoding categorical features → splitting the data → training the Decision Tree → hyperparameter tuning → evaluation → deployment and monitoring**.

Step 1: Understand and inspect the data

First, I would load the dataset using Pandas and understand its structure. I would check the number of rows and columns, data types, missing values, duplicate records, and the distribution of the target variable. For example, features could include **age, blood pressure, cholesterol level, gender, smoking status, and family history**, while the target variable could be Disease, containing values such as 0 = No Disease and 1 = Disease. This step helps us understand what kind of preprocessing is required.

Step 2: Handle missing values

Missing values must be handled before training the model because many Machine Learning algorithms cannot directly work with missing values. First, I would identify which columns contain missing values and how many values are missing. For **numerical features**, missing values can commonly be replaced using the **mean or median**. Median is often useful when the data contains outliers. For **categorical features**, missing values can be replaced with the **mode**, which is the most frequently occurring category, or with a separate category such as "Unknown". If a column has an extremely large number of missing values and does not provide useful information, it may be considered for removal. Importantly, imputation values should be calculated using the training data to avoid **data leakage**.

Step 3: Encode categorical features

Decision Tree models require categorical information to be represented numerically.

Therefore, categorical features such as Gender, Smoking_Status, or Blood_Group need to be

encoded. **One-Hot Encoding** can be used for categories that do not have a natural order. For example, Gender = Male/Female can be converted into numerical indicator columns. If a categorical variable has a meaningful order, such as Risk = Low/Medium/High, an appropriate ordinal encoding can be used. One-Hot Encoding is generally a safe choice when categories have no natural ranking.

Step 4: Split the dataset

After preprocessing, I would divide the dataset into **training and testing sets**. For example, we can use 80% of the data for training and 20% for testing. The training data is used to learn the patterns, while the test data is kept separate and used to check how well the model performs on unseen patients. For a disease prediction problem, I would also consider using **stratified splitting** so that the proportion of disease and non-disease cases remains similar in both datasets.

Step 5: Train the Decision Tree model

Next, I would create a DecisionTreeClassifier from Scikit-learn and train it using the training data. The Decision Tree will learn rules from the patient features to classify patients into disease and non-disease categories. For example, the tree might learn rules such as **"If blood pressure > a particular value and age > a particular value, then the patient has a higher probability of belonging to the disease class."** We can use criteria such as **Gini Impurity or Entropy** for selecting the best splits.

A simple Python example is:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

model.fit(X_train, y_train)
```

Step 6: Tune the hyperparameters

A Decision Tree can easily become too deep and overfit the training data. Therefore, I would tune its **hyperparameters**. Important parameters include max_depth, which controls the maximum depth of the tree; min_samples_split, which specifies the minimum number of samples needed to split a node; and min_samples_leaf, which controls the minimum number of samples that must be present in a leaf node. I can use **GridSearchCV** to test different combinations of these parameters using cross-validation and select the combination that gives the best validation performance.

```
from sklearn.model_selection import GridSearchCV
```

```
parameters = {  
    "max_depth": [3, 5, 7, 10],  
    "min_samples_split": [2, 5, 10],  
    "min_samples_leaf": [1, 2, 5]  
}  
  
grid = GridSearchCV(  
    DecisionTreeClassifier(random_state=42),  
    parameters,  
    cv=5,  
    scoring="recall"  
)  
  
grid.fit(X_train, y_train)  
  
print(grid.best_params_)
```

For healthcare, I would not automatically choose accuracy as the only tuning metric. Depending on the purpose of the system, **recall/sensitivity** can be especially important because missing a patient who actually has the disease (**false negative**) can have serious consequences.

Step 7: Evaluate the model

After selecting the best model, I would evaluate it on the **unseen test dataset**. Accuracy can be calculated, but for healthcare classification it is important to examine several metrics. **Precision** tells us how many patients predicted as having the disease actually have it. **Recall or sensitivity** tells us how many actual disease cases were correctly identified. **F1-score** provides a balance between precision and recall. A **confusion matrix** shows the numbers of true positives, true negatives, false positives, and false negatives. Depending on the medical application, I would also consider **ROC-AUC** and possibly **PR-AUC**, especially if the disease is relatively rare.

```
from sklearn.metrics import accuracy_score  
from sklearn.metrics import classification_report  
from sklearn.metrics import confusion_matrix  
  
y_pred = grid.best_estimator_.predict(X_test)  
  
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Step 8: Check whether the model is suitable for real-world use

Before using the model with real patients, I would check whether its performance is reliable across different patient groups and whether the model is making reasonable predictions. The model should be validated using appropriate healthcare data, monitored after deployment, and reviewed by qualified healthcare professionals. A prediction model should be treated as a **decision-support tool rather than a replacement for medical diagnosis**.

Business value in the real world

A well-designed disease prediction model can provide significant **business and healthcare value**. It can help healthcare organizations identify **high-risk patients earlier**, allowing doctors to prioritize patients who may need additional testing or attention. Early identification can potentially improve patient outcomes and help organizations use medical resources more efficiently. It can also support **preventive healthcare**, reduce unnecessary delays, and help hospitals or healthcare companies plan resources based on patient risk. For example, if the model identifies a patient as high-risk, the healthcare provider can recommend further clinical examination instead of waiting until more serious symptoms appear.

However, the business value depends on the model being **accurate, reliable, fair, interpretable, and clinically validated**. In healthcare, false predictions can have serious consequences, so performance should not be judged only by a single metric such as accuracy.